

# XBoing Screen State Machine: A Z Specification

Formal Model of the Flat Mode Machine and its Shared Resources

July 2026

## Contents

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>2</b> |
| <b>2</b> | <b>Free Types</b>                                     | <b>2</b> |
| <b>3</b> | <b>Screen Sets and the Attract Cycle</b>              | <b>2</b> |
| <b>4</b> | <b>State</b>                                          | <b>3</b> |
| <b>5</b> | <b>Safety Invariants (the intended contract)</b>      | <b>3</b> |
| <b>6</b> | <b>Operations (as coded)</b>                          | <b>4</b> |
| 6.1      | Boot: Presents auto-advances to Intro . . . . .       | 4        |
| 6.2      | Attract auto-advance and the C key . . . . .          | 4        |
| 6.3      | Starting a game — level load succeeds . . . . .       | 5        |
| 6.4      | Starting a game — level load fails silently . . . . . | 5        |
| 6.5      | Space on the title . . . . .                          | 5        |
| 6.6      | Space on the high-score screen . . . . .              | 6        |
| 6.7      | Game over . . . . .                                   | 6        |
| 6.8      | The per-tick rule check . . . . .                     | 7        |
| 6.9      | Bonus completes: next level loads . . . . .           | 7        |
| <b>7</b> | <b>Conjectures: where the invariants break</b>        | <b>7</b> |
| <b>8</b> | <b>Verification summary</b>                           | <b>8</b> |

# 1 Introduction

The XBoing screen machine is *flat*. `struct sdl2_state` (`src/sdl2_state.c`) holds a single `current` mode (`SDL2ST_*`), plus `previous` and a `saved` slot for dialogue push/pop. There is no enclosing “attract” super-state: `Intro`, `Instruct`, `Demo`, `Preview`, `Keys`, `KeysEdit` and `Highscore` are all peer top-level modes. The “attract loop” is a designated *subset* with a constrained transition order (`attract_cycle[]`), i.e. a constraint on a flat machine, not a nesting.

A second, orthogonal nesting axis does exist but is out of scope here: while `current` sits on an attract mode, that mode’s driving *system* runs its own animation phase (`intro_state_t`, `demo_state_t`, ...) held inside `ctx->intro` / `ctx->demo` / `ctx->keys`, not inside `sdl2_state`. Three systems are each shared by two modes (`intro_system` drives `Intro/Instruct`; `demo_system` drives `Demo/Preview`; `keys_system` drives `Keys/KeysEdit`).

Three resources are *shared* between the attract screens and real gameplay: the `game_active` flag, the required-block grid (`block_system_still_active`), and the high-score tables (`hs_global` / `hs_personal`, chosen by `highscore_request_type`). The observed defects all arise because a transition reads a shared resource that a prior transition left in the wrong state. This model names the safety invariants the handlers must preserve and shows which operations break them.

## 2 Free Types

The outer modes. Legacy `BallWait/Wait` (defined but never assigned) are omitted.

$$\begin{aligned} \text{Mode} ::= & mNone \mid mPresents \mid mIntro \mid mInstruct \\ & \mid mDemo \mid mPreview \mid mKeys \mid mKeysedit \\ & \mid mHighscore \mid mBonus \mid mGame \mid mPause \\ & \mid mDialogue \mid mEdit \end{aligned}$$

The shared inner systems (descriptive: the nesting axis noted above).

$$\text{System} ::= sysNone \mid sysIntro \mid sysDemo \mid sysKeys$$

Which high-score table the display is bound to (`highscore_request_type`).

$$\text{Board} ::= bGlobal \mid bPersonal$$

fuzz has no built-in boolean; `ZBOOL` avoids the `ProB/B` keyword clash with `BOOL`.

$$\text{ZBOOL} ::= ztrue \mid zfalse$$

## 3 Screen Sets and the Attract Cycle

The attract screens form the auto-cycle; `Presents` is not part of it.

$$\begin{array}{|l} \hline \text{attractModes} : \mathbb{P} \text{ Mode} \\ \hline \text{attractModes} = \{mIntro, mInstruct, mDemo, mPreview, \\ \quad mKeys, mKeysedit, mHighscore\} \end{array}$$

Attract screens on which `Space` starts a game directly. `Intro` (goes to `Instruct`) and `Highscore` (conditional) are handled separately (`game_input.c:399--423`).

$$\begin{array}{|l} \hline \text{spaceStartModes} : \mathbb{P} \text{ Mode} \\ \hline \text{spaceStartModes} = \{mInstruct, mDemo, mPreview, mKeys, mKeysedit\} \end{array}$$

The cycle order used by the `C` key and each screen’s finish-timer auto-advance (`attract_cycle[]`, `game_callbacks.c:54--68`).

|                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $attractNext : Mode \leftrightarrow Mode$                                                                                                                                                                                   |
| $attractNext = \{ mIntro \mapsto mInstruct, \\ mInstruct \mapsto mDemo, \\ mDemo \mapsto mKeys, \\ mKeys \mapsto mKeysedit, \\ mKeysedit \mapsto mPreview, \\ mPreview \mapsto mHighscore, \\ mHighscore \mapsto mIntro \}$ |

The shared system driving each attract screen (two-to-one sharing).

|                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $driver : Mode \leftrightarrow System$                                                                                                                                                 |
| $driver = \{ mIntro \mapsto sysIntro, \\ mInstruct \mapsto sysIntro, \\ mDemo \mapsto sysDemo, \\ mPreview \mapsto sysDemo, \\ mKeys \mapsto sysKeys, \\ mKeysedit \mapsto sysKeys \}$ |

## 4 State

The flattened screen state. `blocksLoaded` abstracts `block_system_still_active` (required blocks present). `globalHasData` / `personalHasData` record whether each table held rows after the startup read (`game_init.c:327--335`); on an unprivileged install (no `/var/games`) `globalHasData` is `zfalse` while `personalHasData` may be `ztrue`. `boardChoice` is `highscore_request_type`.

|                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------|
| $Screen$                                                                                                                               |
| $mode : Mode \\ gameActive : ZBOOL \\ blocksLoaded : ZBOOL \\ boardChoice : Board \\ globalHasData : ZBOOL \\ personalHasData : ZBOOL$ |

The machine boots into `Presents`: `ctx` is `calloc`'d so `gameActive` is `zfalse` (`game_init.c:231`); no level is loaded; both boards are read from disk (`game_init.c:327--335`) so the data flags reflect what was on disk — left unconstrained here because they are platform-dependent; and `boardChoice` defaults to `bGlobal` (`game_init.c:340`).

|                                                                                                 |
|-------------------------------------------------------------------------------------------------|
| $Init$                                                                                          |
| $Screen'$                                                                                       |
| $mode' = mPresents \\ gameActive' = zfalse \\ blocksLoaded' = zfalse \\ boardChoice' = bGlobal$ |

## 5 Safety Invariants (the intended contract)

Stated as observer schemas, deliberately *not* folded into `Screen`: in Z the state predicate constrains every  $\Delta Screen$ , so folding an invariant in would silently *disable* the violating transitions rather than expose them (probccli then reports a spurious deadlock). Instead each contract is machine-checked by model-checking toward its *negation* as a probccli goal:

```
probccli spec.tex -model_check -goal "<negated invariant>"
```

A reachable state matching the negation is a counter-example proving the defect; “No counter example found. ALL states visited” after a fix proves the guard closes it.

A game may only run once its level’s required blocks are on the grid; otherwise the per-tick rule check reads an empty grid and declares the level complete.

|                                                 |       |
|-------------------------------------------------|-------|
| <i>SafeGame</i>                                 | _____ |
| <i>Screen</i>                                   | _____ |
| $mode = mGame \Rightarrow blocksLoaded = ztrue$ |       |

The high-score screen must not display an *empty* board when a populated one is available: if it is bound to an empty global board, the personal board must also be empty (nothing better to show).

|                                                                                                                                                  |       |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <i>SafeHighscore</i>                                                                                                                             | _____ |
| <i>Screen</i>                                                                                                                                    | _____ |
| $(mode = mHighscore$<br>$\quad \wedge boardChoice = bGlobal$<br>$\quad \wedge globalHasData = zfalse)$<br>$\Rightarrow personalHasData = zfalse$ |       |

`game_active` may legitimately stay true on the game-over **Highscore** display (it gates score submission), but it must not propagate to the *other* attract screens; otherwise the **Highscore Space** routing misfires.

|                                                                                            |       |
|--------------------------------------------------------------------------------------------|-------|
| <i>SafeAttract</i>                                                                         | _____ |
| <i>Screen</i>                                                                              | _____ |
| $(mode \in attractModes \wedge mode \neq mHighscore)$<br>$\Rightarrow gameActive = zfalse$ |       |

## 6 Operations (as coded)

### 6.1 Boot: Presents auto-advances to Intro

The machine boots into Presents; when the credits animation finishes, `presents_cb_on_finished` (`game_callbacks.c:596--600`) transitions unconditionally to Intro, entering the attract cycle. Presents is not part of `attract_cycle[]`.

|                                                                                                                                                                                                                    |       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <i>Boot</i>                                                                                                                                                                                                        | _____ |
| $\Delta Screen$                                                                                                                                                                                                    | _____ |
| $mode = mPresents$<br>$mode' = mIntro$<br>$gameActive' = gameActive$<br>$blocksLoaded' = blocksLoaded$<br>$boardChoice' = boardChoice$<br>$globalHasData' = globalHasData$<br>$personalHasData' = personalHasData$ |       |

### 6.2 Attract auto-advance and the C key

Both the finish-timer (`highscore_cb_on_finished` and peers) and the C key advance the cycle. Crucially, this transition *preserves* `gameActive` — the auto-advance from **Highscore** does not clear it (`game_callbacks.c:750--759`). Entering **Highscore** binds the display to `boardChoice` but performs no disk read and no global-to-personal fallback on the attract path (`game_modes.c:1139--1142`).

*AttractAdvance*

$\Delta$ *Screen*

$mode \in \text{dom } \text{attractNext}$   
 $mode' = \text{attractNext}(mode)$   
 $(mode' = mIntro \Rightarrow gameActive' = zfalse)$   
 $(mode' \neq mIntro \Rightarrow gameActive' = gameActive)$   
 $blocksLoaded' = blocksLoaded$   
 $(mode' = mHighscore \wedge gameActive = zfalse$   
 $\quad \wedge globalHasData = zfalse \wedge personalHasData = ztrue$   
 $\quad \Rightarrow boardChoice' = bPersonal)$   
 $(\neg (mode' = mHighscore \wedge gameActive = zfalse$   
 $\quad \wedge globalHasData = zfalse \wedge personalHasData = ztrue)$   
 $\quad \Rightarrow boardChoice' = boardChoice)$   
 $globalHasData' = globalHasData$   
 $personalHasData' = personalHasData$

### 6.3 Starting a game — level load succeeds

`start_new_game` (`game_modes.c:90--166`) sets `gameActive` and, on a successful load, populates the grid via the `on_add_block` callback. This is the correct path.

*StartNewGameOk*

$\Delta$ *Screen*

$mode \in \text{spaceStartModes}$   
 $mode' = mGame$   
 $gameActive' = ztrue$   
 $blocksLoaded' = ztrue$   
 $boardChoice' = boardChoice$   
 $globalHasData' = globalHasData$   
 $personalHasData' = personalHasData$

### 6.4 Starting a game — level load fails silently

If the level file cannot be resolved or parsed (`game_modes.c:128--136`) `start_new_game` only warns to `stderr` and still enters `Game` with the grid left cleared.

*StartNewGameFail*

$\Delta$ *Screen*

$mode \in \text{spaceStartModes}$   
 $mode' = mIntro$   
 $gameActive' = zfalse$   
 $blocksLoaded' = zfalse$   
 $boardChoice' = boardChoice$   
 $globalHasData' = globalHasData$   
 $personalHasData' = personalHasData$

### 6.5 Space on the title

*IntroSpace*

$\Delta$ Screen

```
mode = mIntro
mode' = mInstruct
gameActive' = gameActive
blocksLoaded' = blocksLoaded
boardChoice' = boardChoice
globalHasData' = globalHasData
personalHasData' = personalHasData
```

## 6.6 Space on the high-score screen

The `Space` handler routes on `gameActive` and clears it here (`game_input.c:411--423`): a live game-over returns to `Intro`; otherwise it starts a game.

*HighscoreSpaceReturn*

$\Delta$ Screen

```
mode = mHighscore
gameActive = ztrue
mode' = mIntro
gameActive' = zfalse
blocksLoaded' = blocksLoaded
boardChoice' = boardChoice
globalHasData' = globalHasData
personalHasData' = personalHasData
```

*HighscoreSpaceStart*

$\Delta$ Screen

```
mode = mHighscore
gameActive = zfalse
mode' = mGame
gameActive' = ztrue
blocksLoaded' = ztrue
boardChoice' = boardChoice
globalHasData' = globalHasData
personalHasData' = personalHasData
```

## 6.7 Game over

`game_rules_ball_died` (`game_rules.c:294--296`) leaves `gameActive` true and shows `Highscore`.

*GameOver*

$\Delta$ Screen

```
mode = mGame
mode' = mHighscore
gameActive' = ztrue
blocksLoaded' = blocksLoaded
(globalHasData = zfalse  $\wedge$  personalHasData = ztrue
 $\Rightarrow$  boardChoice' = bPersonal)
( $\neg$  (globalHasData = zfalse  $\wedge$  personalHasData = ztrue)
 $\Rightarrow$  boardChoice' = bGlobal)
globalHasData' = globalHasData
personalHasData' = personalHasData
```

## 6.8 The per-tick rule check

Every Game tick, `game_rules_check` (`game_rules.c:330--340`) sends the machine to Bonus the instant no required blocks remain. It is *unconditional* — no guard that a level was actually loaded.

|                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>GameRulesToBonus</i><br>$\Delta\text{Screen}$<br><br><i>mode</i> = <i>mGame</i><br><i>blocksLoaded</i> = <i>zfalse</i><br><i>mode'</i> = <i>mBonus</i><br><i>gameActive'</i> = <i>gameActive</i><br><i>blocksLoaded'</i> = <i>blocksLoaded</i><br><i>boardChoice'</i> = <i>boardChoice</i><br><i>globalHasData'</i> = <i>globalHasData</i><br><i>personalHasData'</i> = <i>personalHasData</i> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6.9 Bonus completes: next level loads

On bonus completion `bonus_cb_on_finished` advances to the next level (`game_rules_next_level`) and re-enters Game with the new level's blocks loaded (`game_callbacks.c:671--678`). Modelled so Bonus is not a terminal (deadlocking) mode.

|                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>BonusToGame</i><br>$\Delta\text{Screen}$<br><br><i>mode</i> = <i>mBonus</i><br><i>mode'</i> = <i>mGame</i><br><i>blocksLoaded'</i> = <i>ztrue</i><br><i>gameActive'</i> = <i>gameActive</i><br><i>boardChoice'</i> = <i>boardChoice</i><br><i>globalHasData'</i> = <i>globalHasData</i><br><i>personalHasData'</i> = <i>personalHasData</i> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 7 Conjectures: where the invariants break

Each defect is a reachable state that violates one safety schema. A model checker (`probcli`) explores `Init` plus the operations and reports the trace; the prose states the trace and the missing guard.

### Defect 2 — empty high-score board in the attract cycle (PROVEN, FIXED)

The boards *are* read at startup, so “never loaded” is not the cause. On an unprivileged install `globalHasData = zfalse` and `personalHasData = ztrue`, and `boardChoice = bGlobal` from `Init`. Goal-directed model checking of the negated `SafeHighscore` reached `mode = mHighscore`, `gameActive = zfalse` still bound to the empty global board by pure attract-cycling (*Init* ; *Boot* ; *AttractAdvance\**), the machine-checked counter-example for the reported symptom.

**Fix.** `mode_highscore_enter` (`src/game_modes.c`) now falls back to `bPersonal` on the attract path (`!game_active`) when the global board is empty (`highscore_io_count(&hs_global) == 0`) and the personal board has scores. The game-over path keeps its own submit-based fallback (`game_modes.c:1134`). Both entry paths are modelled here in `AttractAdvance` and `GameOver`; re-running the goal now reports *No counter example found. ALL states visited* (138 states), so the fixed operations cannot reach the violating state. Regression: `test_highscore_attract_falls_back_to_personal` (`tests/test_integration_modes.c`) plus `highscore_io_count` unit tests.

### Defect 3 — first Space advances the loop (PROVEN, FIXED)

In a *pure* attract cycle `gameActive` is always `zfalse` (set only by `StartNewGame*`, cleared only by the `Highscore Space` handler), so `IntroSpace` (`Intro` → `Instruct`) is the *intended* PR #172 behavior, not a bug. The genuine violation requires a prior game over: goal-directed model checking of the negated `SafeAttract`

found the 7-step trace ... ; *GameOver* ; *AttractAdvance* carrying `gameActive = ztrue` out of *Highscore* into *Intro* (*SafeAttract* violated at *Intro*); the next time the cycle reaches *Highscore*, *HighscoreSpaceReturn* fires and the first *Space* goes to *Intro* instead of starting a game.

**Fix.** Every attract advance out of *Highscore* — the finish-timer (`highscore_cb_on_finished`), the C cycle key (`game_input.c`), and the *Space* return — lands on *Intro*, since `attract_next(HIGHSCORE) == INTRO`. A single clear in `mode_intro_enter` (`src/game_modes.c`) therefore closes all three leak paths at one site; modelled here as *AttractAdvance* setting `gameActive' = zfalse` whenever `mode' = mIntro`. Re-running the goal now reports *No counter example found. ALL states visited* (90 states), and no non-*Highscore* attract mode is reachable with `gameActive = ztrue`. Regression: `test_highscore_autoadvance_clears_game_active` (timer path) and `test_highscore_c_cycle_clears_game_active` (C key). See ADR-055.

## Defect 1 — Space lands in Bonus with an empty grid (PROVEN, FIXED)

*StartNewGameOk* establishes `blocksLoaded`, so the model exonerates the *happy-path* *Space*-to-game edge. Goal-directed model checking of the negated *SafeGame* reached `mode = mGame`, `blocksLoaded = zfalse` via *StartNewGameFail* — a silent level-load failure (`start_new_game` ignored the load result and entered *Game* with the grid cleared); the next *GameRulesToBonus* tick then dropped to *Bonus*.

**Fix.** `start_new_game` (`src/game_modes.c`) now checks `level_system_load_file` and returns failure without setting `game_active`; `mode_game_enter` refuses *Game* and returns to *Intro*. This preserves the original's guarantee — `original/file.c:142--146` (*SetupStage* exits via *ShutDown* on a *ReadNextLevel* failure) never let gameplay run on an unloaded grid — modernising only the mechanism (mode-refusal, not process exit). Modelled as *StartNewGameFail* now targeting `mIntro` with `gameActive = zfalse`; the goal is unreachable (76 states, *No counter example found*). Regression: `test_game_aborts_on_level_load_failure`. See ADR-056.

## 8 Verification summary

Each invariant was machine-checked by model-checking toward its negation as a `probcli` goal. In the as-coded model all three goals were reachable (counter-example traces above); after the fixes all three are unreachable in one exhaustive pass:

- *SafeHighscore* — attract *Highscore* falls back to the populated personal board when the global board is empty (`mode_highscore_enter`; ADR-054).
- *SafeAttract* — `game_active` is cleared on entering the title, so it cannot leak into the attract screens (`mode_intro_enter`; ADR-055).
- *SafeGame* — a failed level load refuses *Game* and returns to the title instead of entering with an empty grid (`start_new_game` / `mode_game_enter`; ADR-056).

```
probccli spec.tex -model_check -goal "<negated invariant>"
=> No counter example found. ALL states visited.
```

The mode suite previously force-transitioned to each mode and ticked for *no-crash* only, asserting nothing about the shared resources. Each invariant is now both a re-proven model postcondition and a red→green regression test, named in its per-defect section above.